

Zero Knowledge

Course Leads:

Sara Magaziotis-Ginori

Olivia Li

Isaac Oh

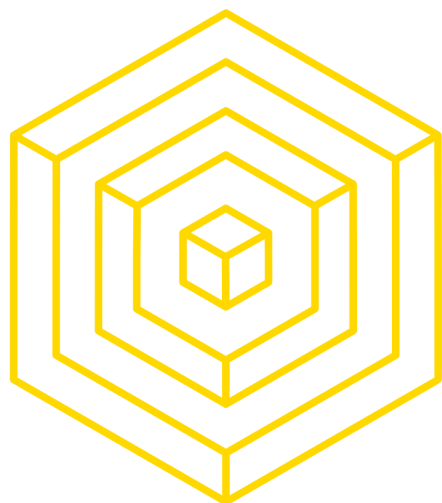
Aman Shah

Elson Liu

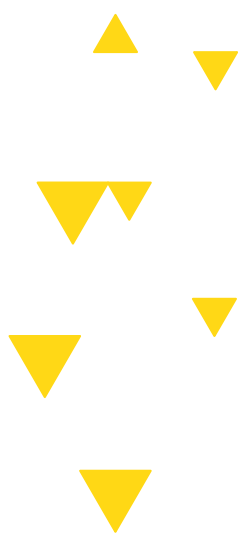
Fujia Wang



BLOCKCHAIN
AT BERKELEY



O Announcements

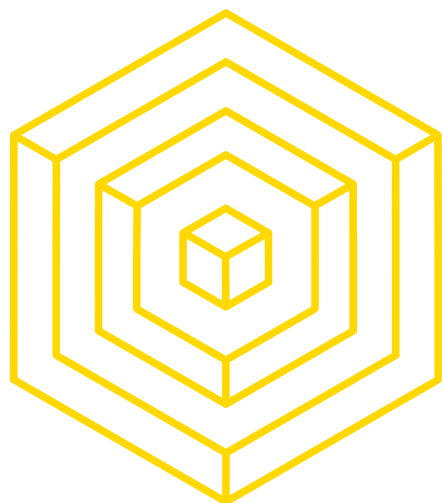




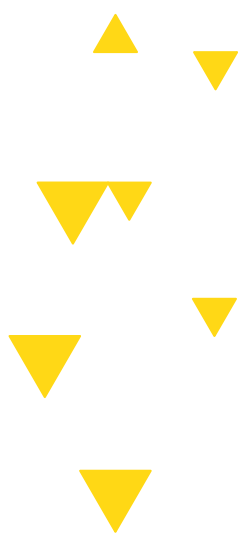
Announcements

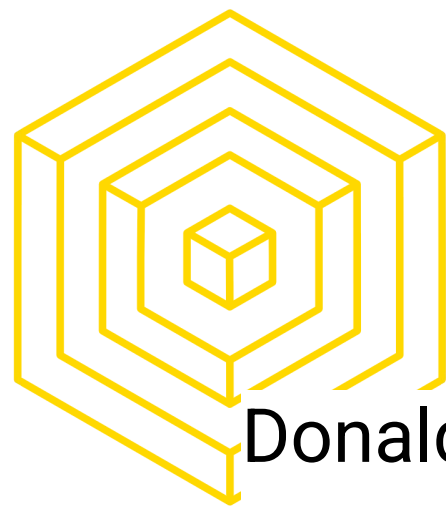
- HW 5 is due today!
- New Lecture! No HW for today!

Source: <https://cointelegraph.com/explained/zero-knowledge-proofs-explained>



O This week in crypto





This week in crypto

Donald Trump, via Truth Social, announced a 100% tariff on Chinese goods in response to the rare earth metals announcement.

Business | decrypt

Morning Minute: \$19B Wiped Out in Crypto's Biggest Liquidation Ever



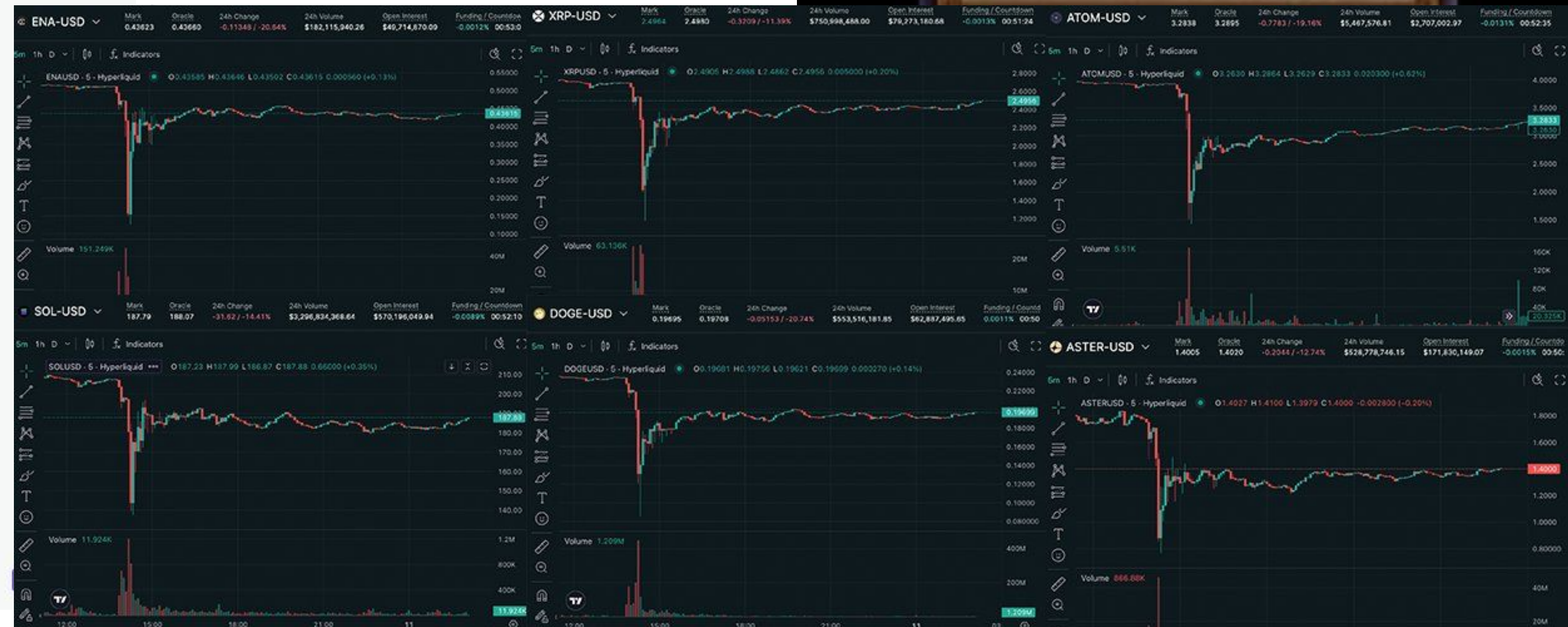
Aaron Slodov @aphysicist · Oct 9

CHINA JUST WEAPONIZED THE ENTIRE RARE EARTH SUPPLY CHAIN

get used to reading chinese MOFCOM bulletins because they're writing american industrial policy now.

announced today: any product containing >0.1% chinese rare earth materials needs beijing's approval before re-export to

[Show more](#)

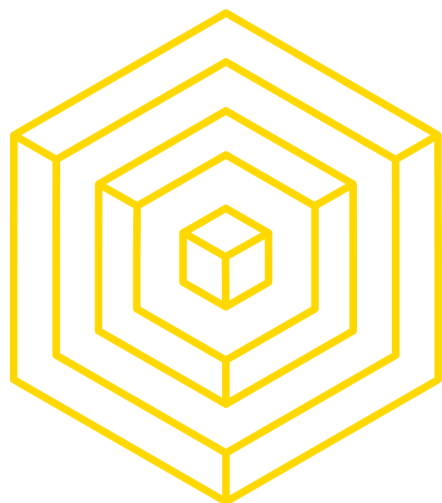


<https://arc.net/l/quote/lpqluzfs>

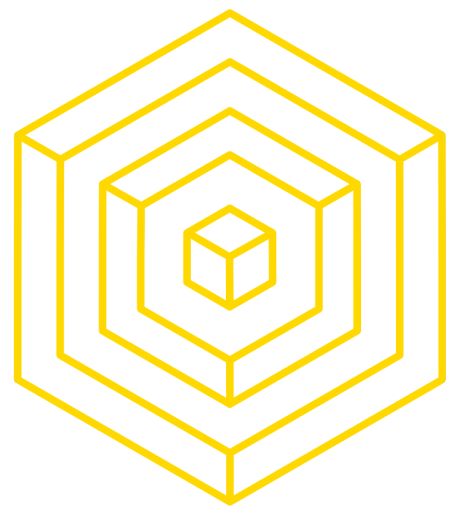


LECTURE OUTLINE

- 1 zk and Cryptography Review
- 2 zk further explained / SNARKS
- 3 zk rollups
- 4 Other use cases / this week news updates



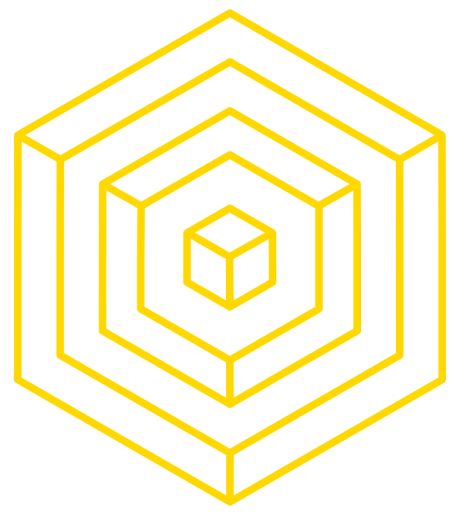
1 ZK Review



What is ZK?

- A way to prove that a statement is true without having to reveal the information you used to prove it
- Three coloring problem
- How can I prove I have a valid coloring without revealing the coloring itself

Source: <https://cointelegraph.com/explained/zero-knowledge-proofs-explained>



How does it work?

- Agree on the map
- Prover randomly chooses a color for each vertex and encrypt it
- Verifier chooses a vertex, the prover then reveals the color of this vertex
- Now the prover can show that the nodes adjacent to the revealed vertex have different colors, this is only possible because we can show a different in the encryptions for the verifier to verify
- These last three steps are repeated until the verifier becomes confident in the provers assertion
- Chance of lying after n rounds is $(e-1 / e)^n$

Source: <https://cointelegraph.com/explained/zero-knowledge-proofs-explained>



How does it work?

- Since this problem is NP complete, all problems in NP can be reduced to it
- NP means solution to a problem can be found and verified in polynomial time
- NP complete means all problems in NP can be reduced to this problem
- Therefore the 3 coloring example shows all problems in NP can be approached with this zero knowledge approach

Source: https://fisher.wharton.upenn.edu/wp-content/uploads/2020/09/Thesis_Terrence-Jo.pdf



Applications

- Zk proofs can allow for private transacting on platforms like Z-Cash
- Zk rollups
- Starknet, zkSync, and Polygon zkEVM
- Zk rollups help with scalability
- Rollups group together transactions before publishing them to the L1 chain
- Rollup operators produce validity proofs for the transactions
- zk SNARKs or zk STARKs are used to verify the validity and the transaction batch is published to the chain if valid

Source: <https://cointelegraph.com/explained/zero-knowledge-proofs-explained>



Cryptography - Hash Functions

- Hash functions are mathematical functions that convert data into a fixed-length character string. They have the following properties:
 - Deterministic - the same input will always produce the same hash
 - Efficient - output hash is efficient to compute
 - Invertible - it is practically impossible to reverse the hash function, so no one can get the input from the output hash
 - Collision resistant - it is computationally infeasible to find two inputs that hash to the same value

Source: https://en.wikipedia.org/wiki/Public-key_cryptography



Common Hash Functions

- D5 (Message Digest Algorithm 5):
 - Produces a 128-bit hash value. Use is discouraged as it is considered weak due to vulnerabilities
- SHA-1 (Secure Hash Algorithm 1):
 - Initially designed for security, SHA-1 is now deprecated for cryptographic use due to vulnerabilities. SHA-256 and SHA-3 are more secure alternatives
- SHA-256 (Secure Hash Algorithm 256-bit):
 - Part of the SHA-2 family, it produces a 256-bit hash value and is widely used for secure applications. One of the most well known and used hash functions
- Blake2:
 - A cryptographic hash function that is fast and secure, often used in applications like password hashing and digital signatures

Source: https://en.wikipedia.org/wiki/Public-key_cryptography



Hash Function Applications

- There are many common uses for hash functions, especially in security
 - Checksums - allow users to verify the integrity of data after transmission. If the data has been tampered with, it will hash to a different value, thus alerting the user
 - Digital signatures - allow users to attest that they sent a certain message
 - Password hashing - hashes allow you to store passwords securely, since you can store the its hash instead of plaintext
 - Hash tables/maps - data structure that allows efficient lookup and retrieval

Source: https://en.wikipedia.org/wiki/Public-key_cryptography



Encryption & Cryptographic Keys

- Encryption is a fundamental technique in information security and is used to protect sensitive data during transmission and storage
- Encryption is the process of converting readable data (plaintext) into an unreadable format (ciphertext) to secure it from unauthorized access
- The reverse process, converting ciphertext back to plaintext, is called decryption.
- This process relies heavily on cryptographic keys

Source: https://en.wikipedia.org/wiki/Public-key_cryptography



Public Key Cryptography - Overview

- Public key (asymmetric) encryption - a method for secure communication that uses a pair of keys to ensure security over an insecure channel
- Advantages:
 - Key distribution - keys can now be openly shared
 - Confidentiality - even if the public key is intercepted, no ciphertext can be decrypted since the private key is needed
 - Digital signatures - public key pairs can be used to verify the sender's identity and integrity of the message
 - Authentication - allows for the use of digital certificates

Source: https://en.wikipedia.org/wiki/Public-key_cryptography



Public Key Cryptography - Encryption

- Sender (User A) uses the recipient's (User B) public key to encrypt the message. Only the corresponding private key, known only to User B, can decrypt the message.
- For example:
 - User A wants to send a confidential message to User B
 - User B shares their public key openly
 - User A encrypts the message using User B's public key
 - The encrypted message can only be decrypted by User B's private key

Source: https://en.wikipedia.org/wiki/Public-key_cryptography



Public Key Cryptography - Decryption

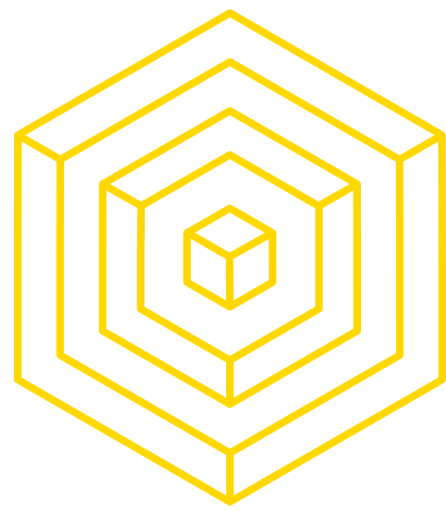
- Private Key Decryption (Decryption Process):
- The recipient (User B) uses their private key to decrypt the message. The private key is kept secret and is never shared.
- For example:
 - User B receives the encrypted message.
 - User B uses their private key to decrypt the message and recover the original plaintext.

Source: https://en.wikipedia.org/wiki/Public-key_cryptography



2

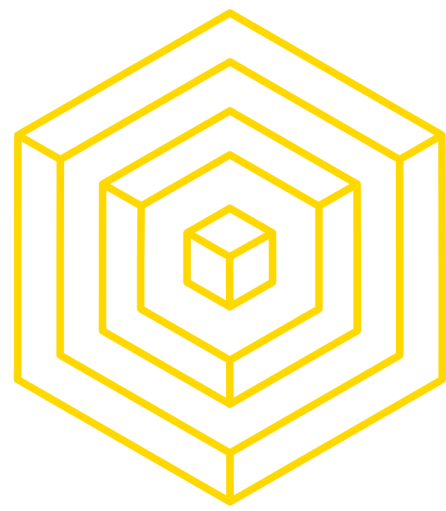
ZK Further Explained



What is ZK?

- Interactive ZK-proofs
- Interactive zero-knowledge proofs require back-and-forth communication between the prover and verifier
- Non-interactive ZK-proofs
- Non-interactive zero-knowledge proofs provide a compact proof that can be verified in a single step
- Statistical ZK-proofs
- Statistical zero-knowledge proofs offer computational soundness with a small probability of error

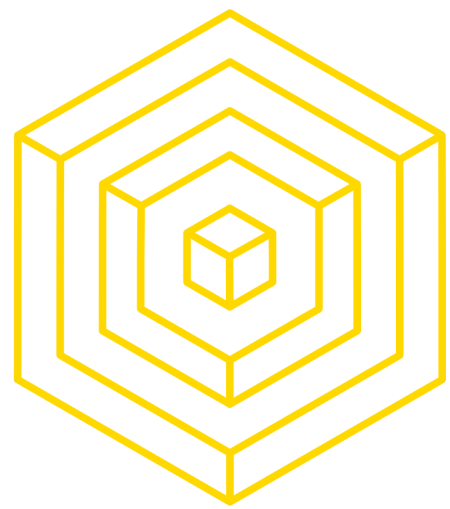
Source: <https://cointelegraph.com/explained/zero-knowledge-proofs-explained>



What is ZK?

- Proof-of-knowledge (PoK)
- PoK is a subclass of ZK-proofs that shows that the prover possesses specific knowledge related to the statement
- Proofs of shuffle and range
- These ZK-proofs are used in electronic voting and privacy-preserving transactions
- Sigma protocols
- Sigma protocols are a class of ZK-proofs that involve three steps: commitment, challenge and response

Source: <https://cointelegraph.com/explained/zero-knowledge-proofs-explained>



Guarantees

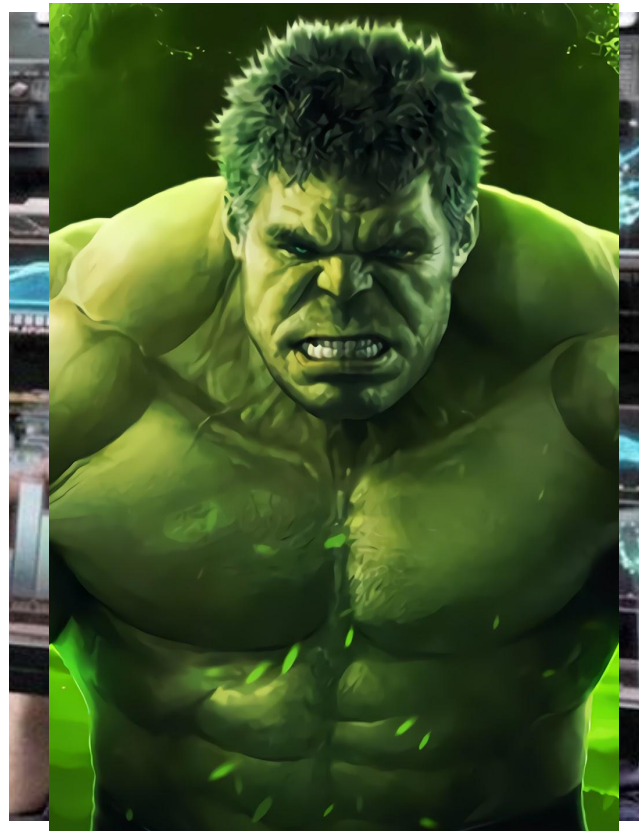
- Completeness - The verifier will be convinced of true statements by the prover
- Soundness - The prover can only convince the verifier of statements that are actually true, with very small probability of it not being true
- Zero-Knowledge - No knowledge of the solution/state is leaked except for the fact that the statement is true

Source: https://fisher.wharton.upenn.edu/wp-content/uploads/2020/09/Thesis_Terrence-Jo.pdf



The story – what is zk-SNARK

VERIFIER



**Brink
Banner**

$p(x)$



PROVER



Jeydel

Zero-Knowledge Succinct
Non-Interactive Argument
of Knowledge

- Completeness
- Soundness
- Non-interactive
- Zero-Knowledge



Day 1: simple proof

$$p(x) = x^2 + x + 1$$

$x = 1$
 $p(1) = 3$



Hulk

$x = 1$

$p(1) = 3$ ✓





Day 2: factorized proof

$$p(x) = t(x) \cdot h(x) \quad p(x) = x^3 - 3x^2 + 2x$$

$$h(x) = x \quad t(x) = x^2 - 3x + 2$$

Thought bubble:

$$x = 3$$

$$t(3) = 2$$

Right arrow:

$$x = 3$$

Left arrow:

$$h(3) = 3 \text{ and } p(3) = 6$$

$$p(x) = t(x) \cdot h(x)$$

$$6 = 2 \times 3$$



$$h(x) = \frac{p(x)}{t(x)}$$

$$\begin{array}{r} x \\ x^2 - 3x + 2 \overline{) x^3 - 3x^2 + 2x} \\ \underline{-x^3 + 3x^2 - 2x} \\ 0 \end{array}$$

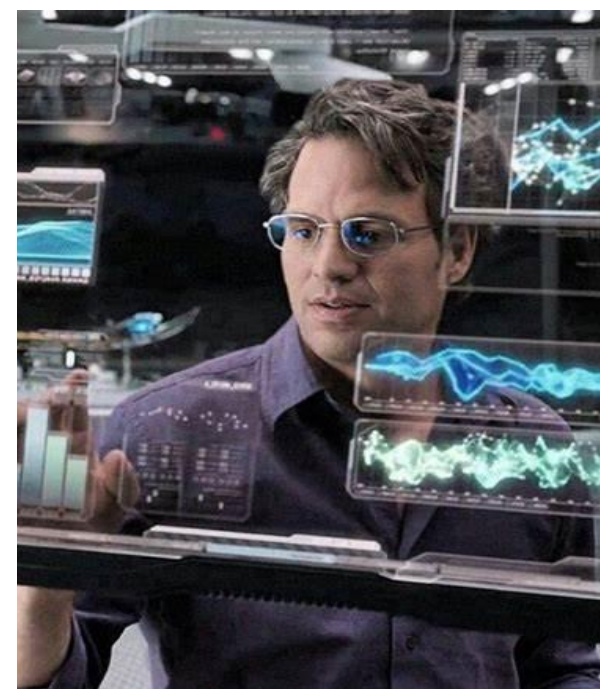
Thought bubble:

$$h(3) = 3$$

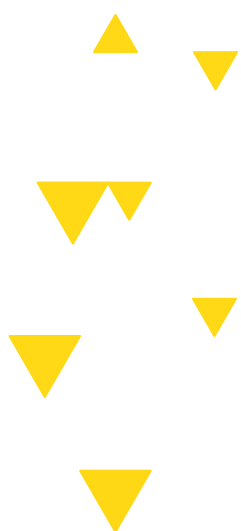
$$p(3) = 6$$



Jekyll



Bruce



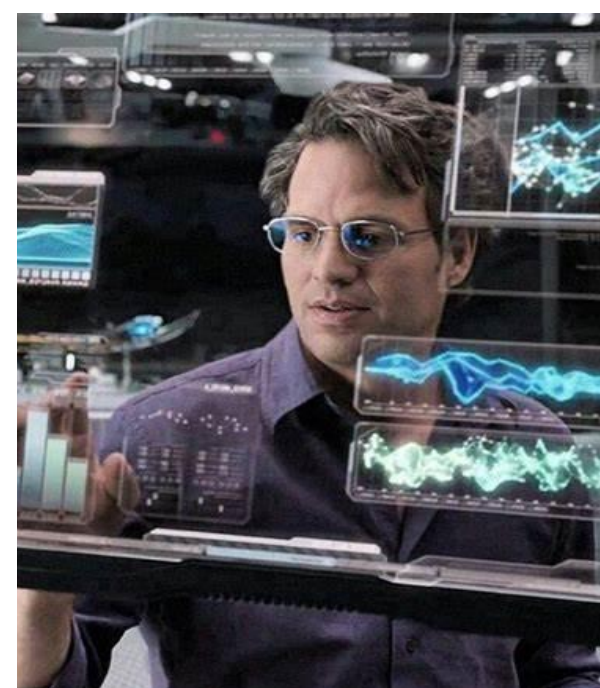


Day 2: factorized proof

$$p(x) = x^3 - 3x^2 + 2x$$

$$t(x) = x^2 - 3x + 2$$

$x = 3$
 $t(3) = 2$



Bruce

$x = 3$

$h(x) = 9$ and $p(x) = 18$

$18 = 2 \times 9$

$$\begin{array}{r} x \\ x^2 - 3x + 2 \overline{) x^3 - 3x^2 + 2x} \\ \underline{-x^3 + 3x^2 - 2x} \\ 0 \end{array}$$

$$p(x) = t(x) \cdot h(x)$$

$t(3) = 2$
 $h(x) = 9$
 $p(x) = 18$



Hyde



Day 3: Strong Homomorphic Encryption

Plaintext

$$E[v] = g^v \pmod n$$

Base

$$p(x) = x^3 - 3x^2 + 2x$$

$$E[p(x)] = \left(g^{x^3}\right)^1 \cdot \left(g^{x^2}\right)^{-3} \cdot \left(g^x\right)^2$$

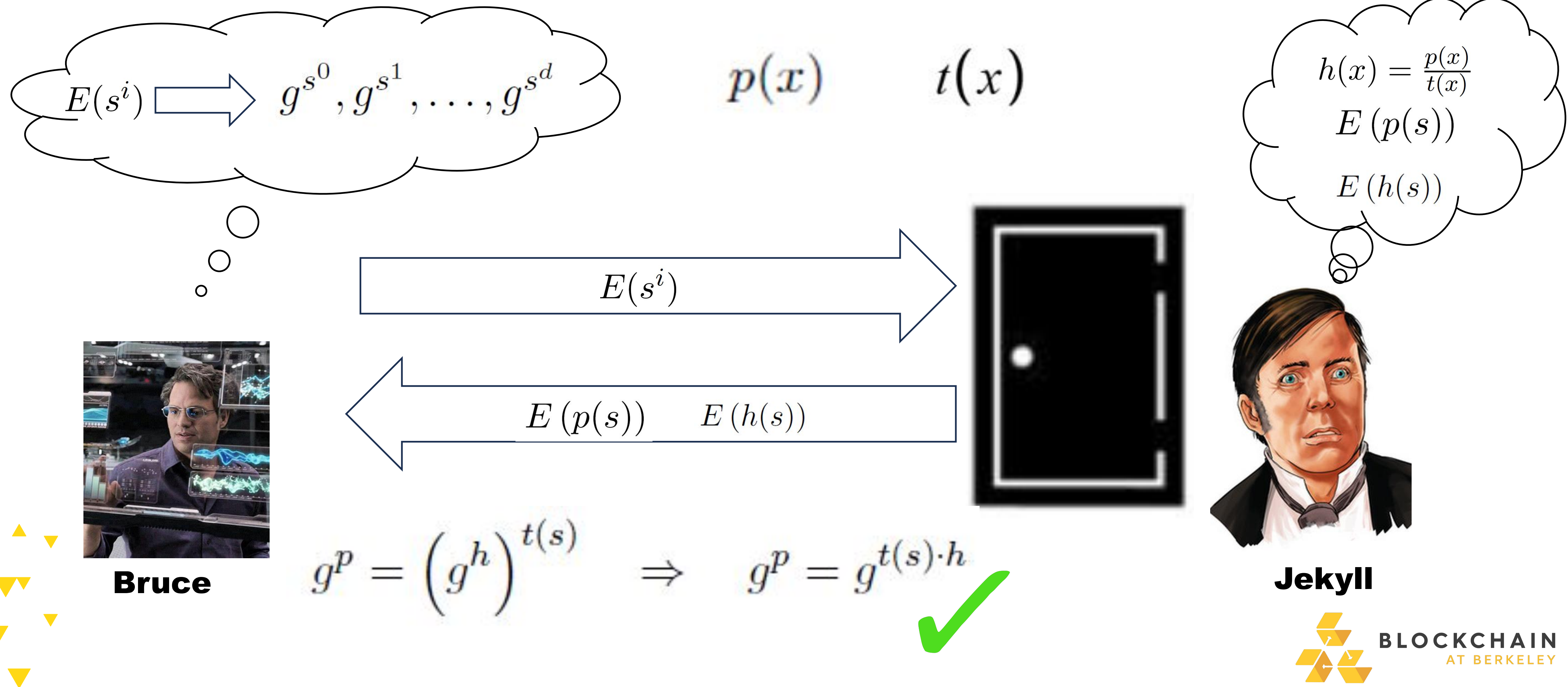
$$g^{1x^3} \cdot g^{-3x^2} \cdot g^{2x}$$

$$g^{x^3 - 3x^2 + 2x}$$

• Secret Key: $E(s^i) = g^{s^i}$

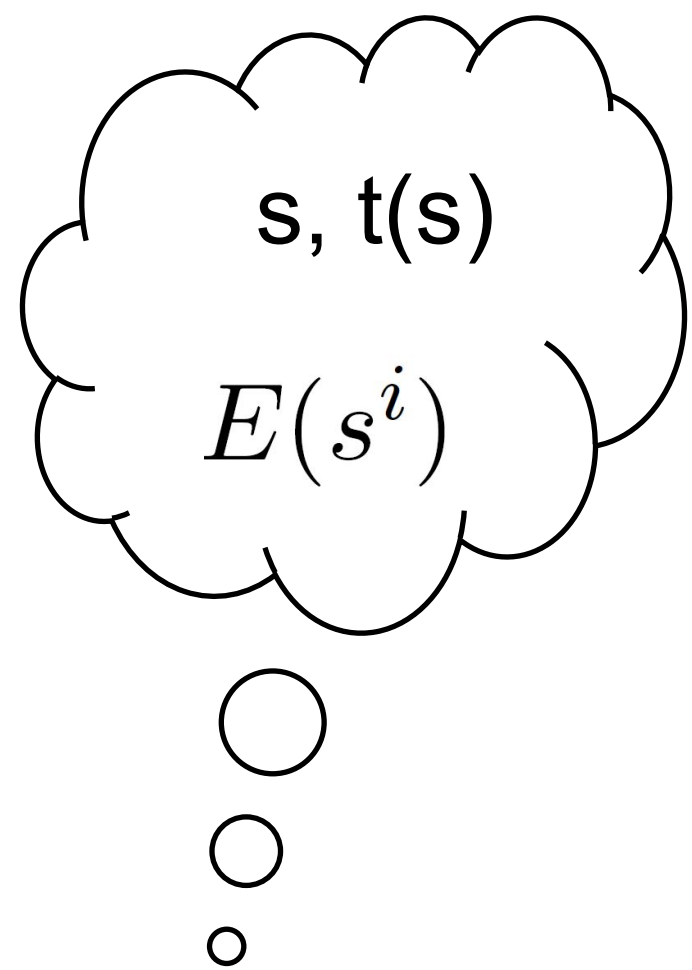


Day 3: Strong homomorphic encryption





Day 3: Strong homomorphic encryption

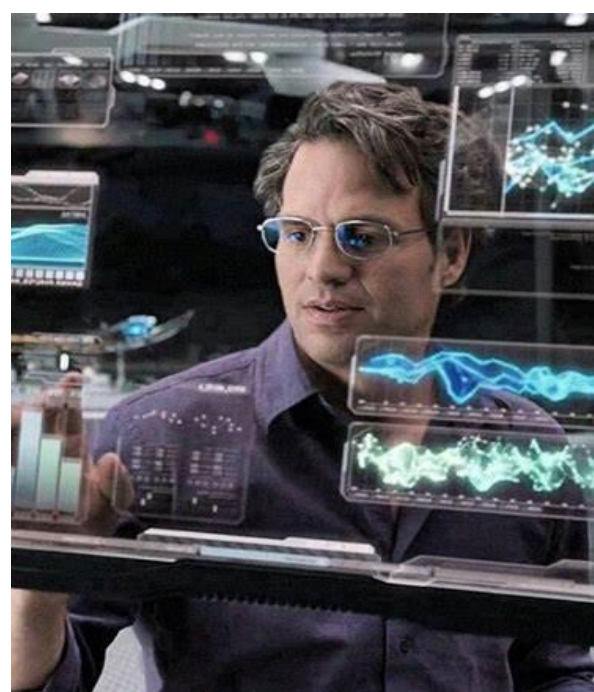
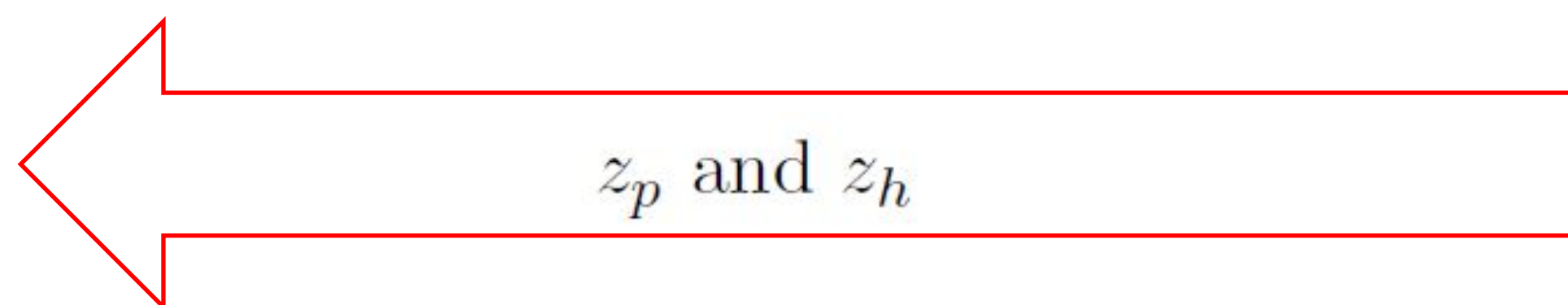
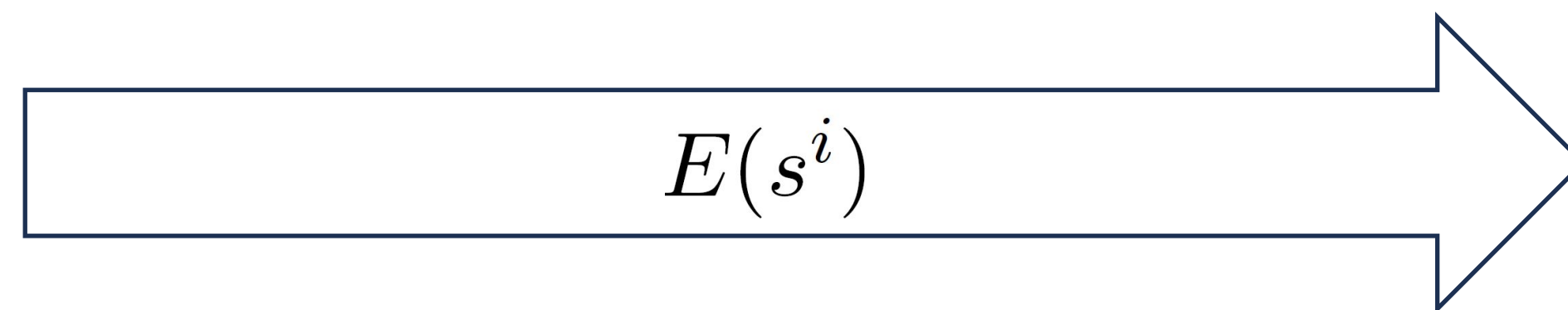


$$t(x) \quad g$$

$$z_h = g^r$$

$$g^{t(s)}$$

$$z_p = (g^{t(s)})^r$$



Bruce



Hyde

$$z_p = (z_h)^{t(s)} \quad \checkmark$$

$$c_n x^n + \dots + c_1 x^1 + c_0 x^0$$



Day 4: Polynomial restriction

$$p(x) = t(x) \cdot h(x)$$

α -shift

g^s a



Bruce

$$(g^s, g^{\alpha \cdot s}) \pmod{n}$$

$$((g^s)^c, (g^{\alpha \cdot s})^c)$$

c

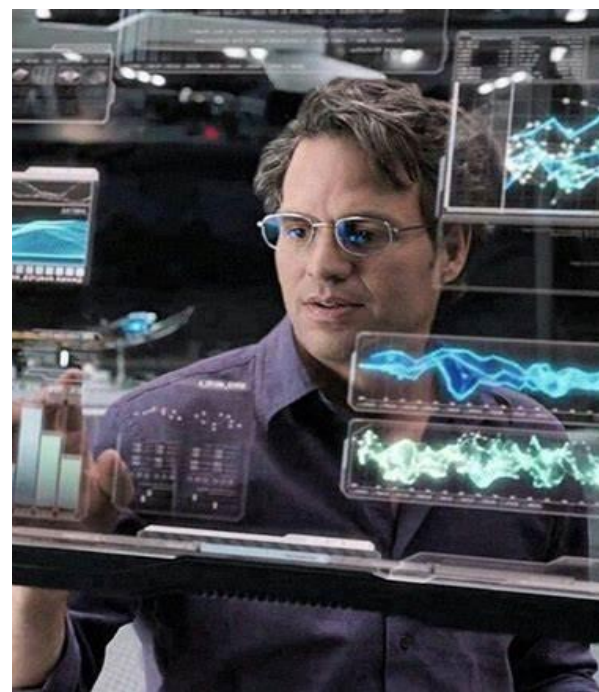
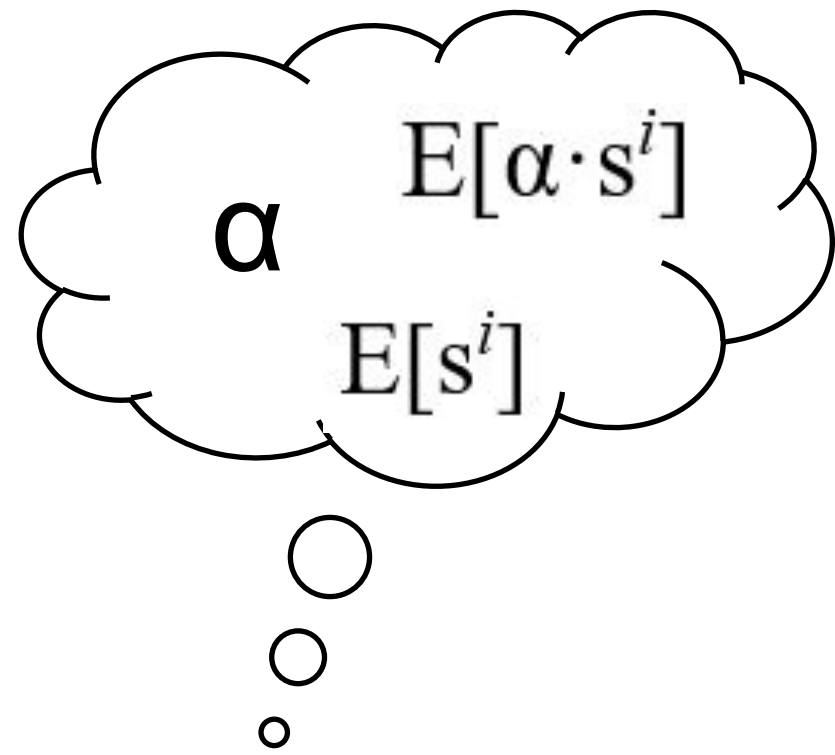


Jekyll

$$(g^{c \cdot s})^\alpha = g^{\alpha \cdot c \cdot s}$$

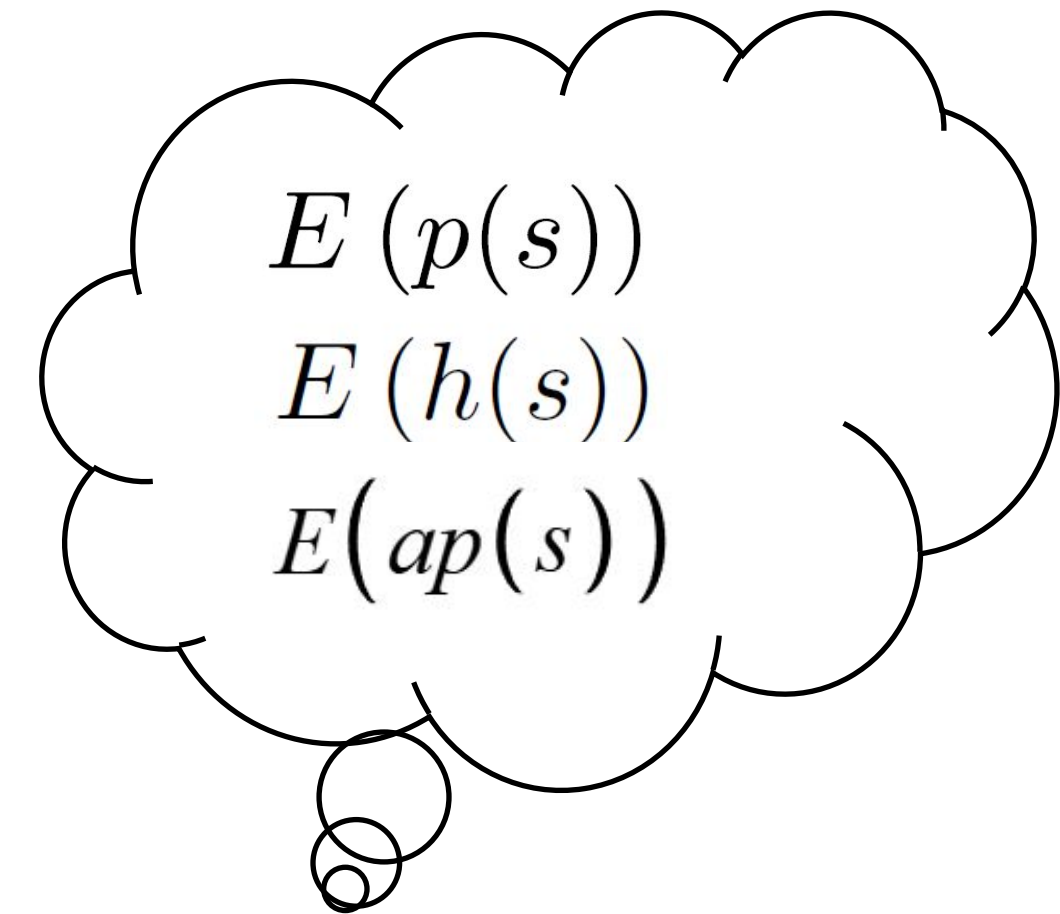
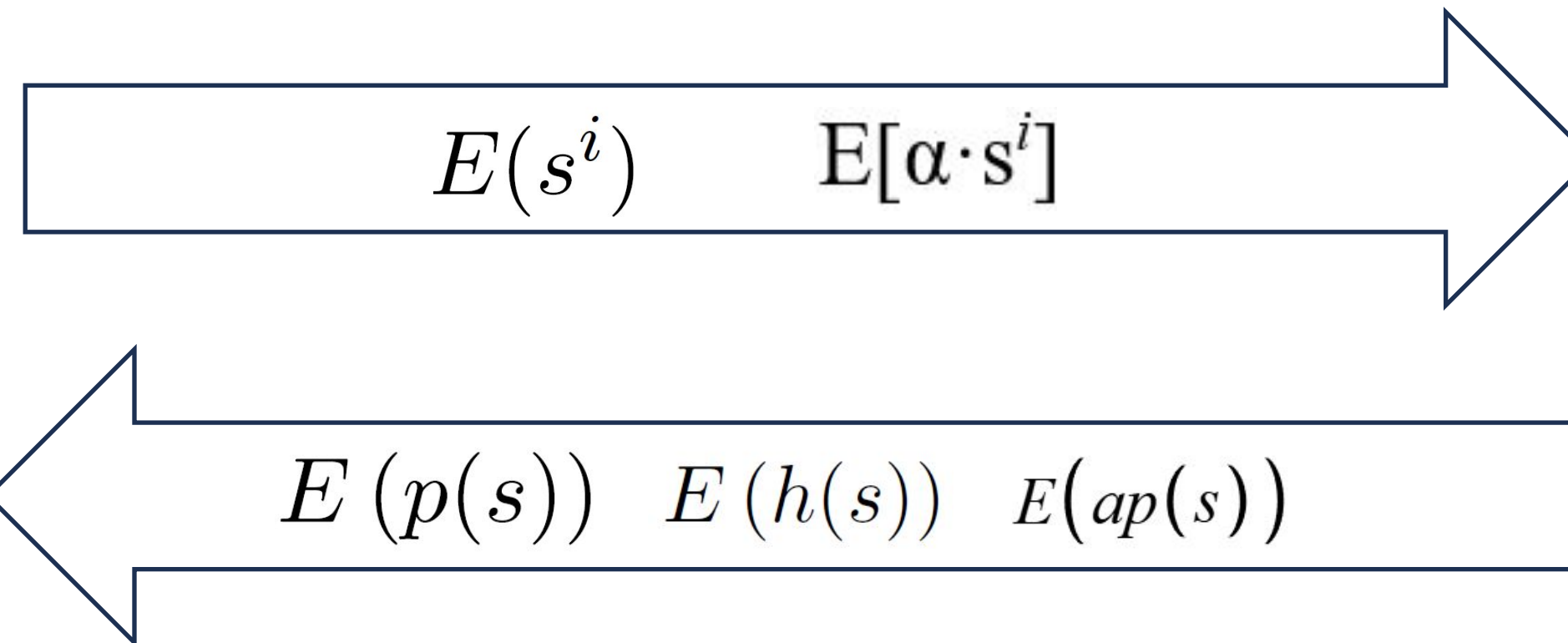


Day 4: Polynomial restriction / α -shift



Bruce

$$p(x) = t(x) \cdot h(x)$$



Jekyll

$$g^p = (g^h)^{t(s)} \Rightarrow g^p = g^{t(s) \cdot h} \checkmark$$

$$E(p(s))^a = E(ap(s)) \checkmark$$



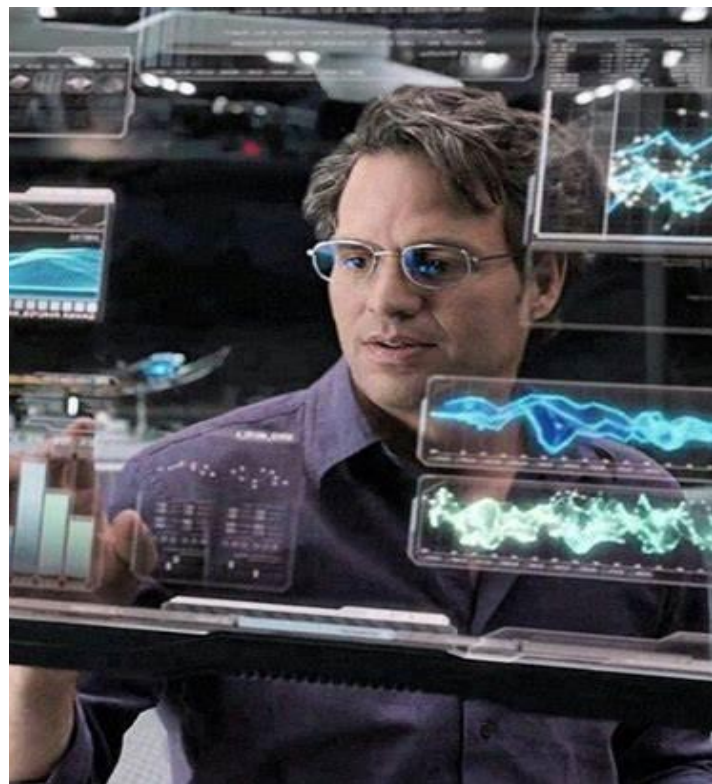
Day 5: cryptographic pairing

$$e(g^a, g^b) = a^g \cdot b^g = (ab)^g$$





Day 6: composite CRS



Bruce

$$\left(g^{s_A^i}, g^{\alpha_A}, g^{\alpha_A s_A^i} \right)$$

Common Reference String (CRS)

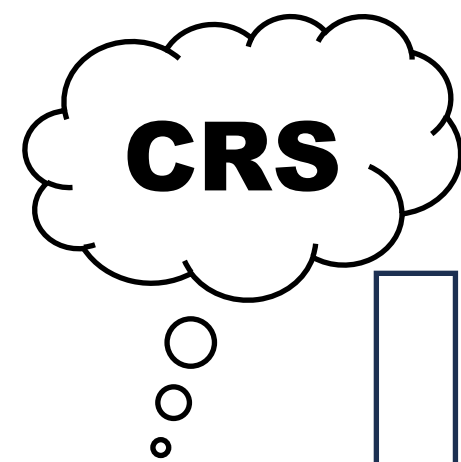


Tony

$$\left(\left(g^{s_A^i} \right)^{s_B^i}, \left(g^{\alpha_A} \right)^{\alpha_B}, \left(g^{\alpha_A s_A^i} \right)^{\alpha_B s_B^i} \right) = \left(g^{(s_A s_B)^i}, g^{\alpha_A \alpha_B}, g^{\alpha_A \alpha_B (s_A s_B)^i} \right)$$



Day 7: zk-SNARK!



CRS

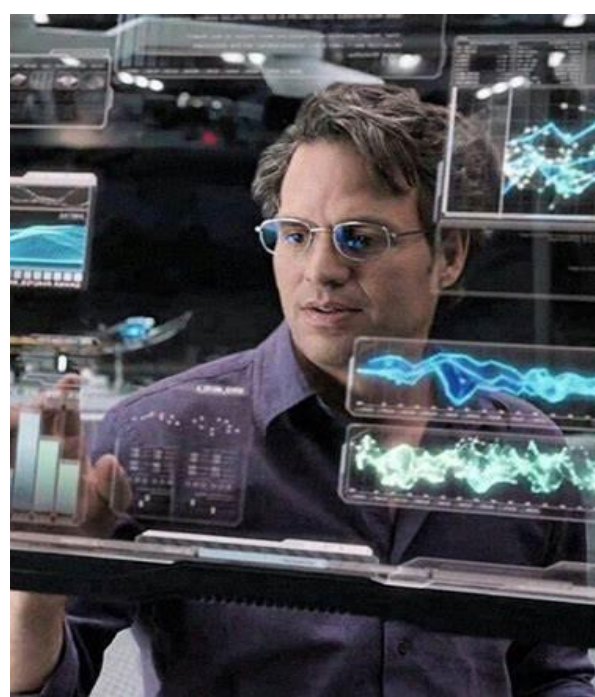
Proving Keys

$E[\alpha \cdot s^i] \quad E(s^i)$

$$p(x) = t(x) \cdot h(x)$$

Verification Keys

$E[\alpha] \quad E[t(s)]$

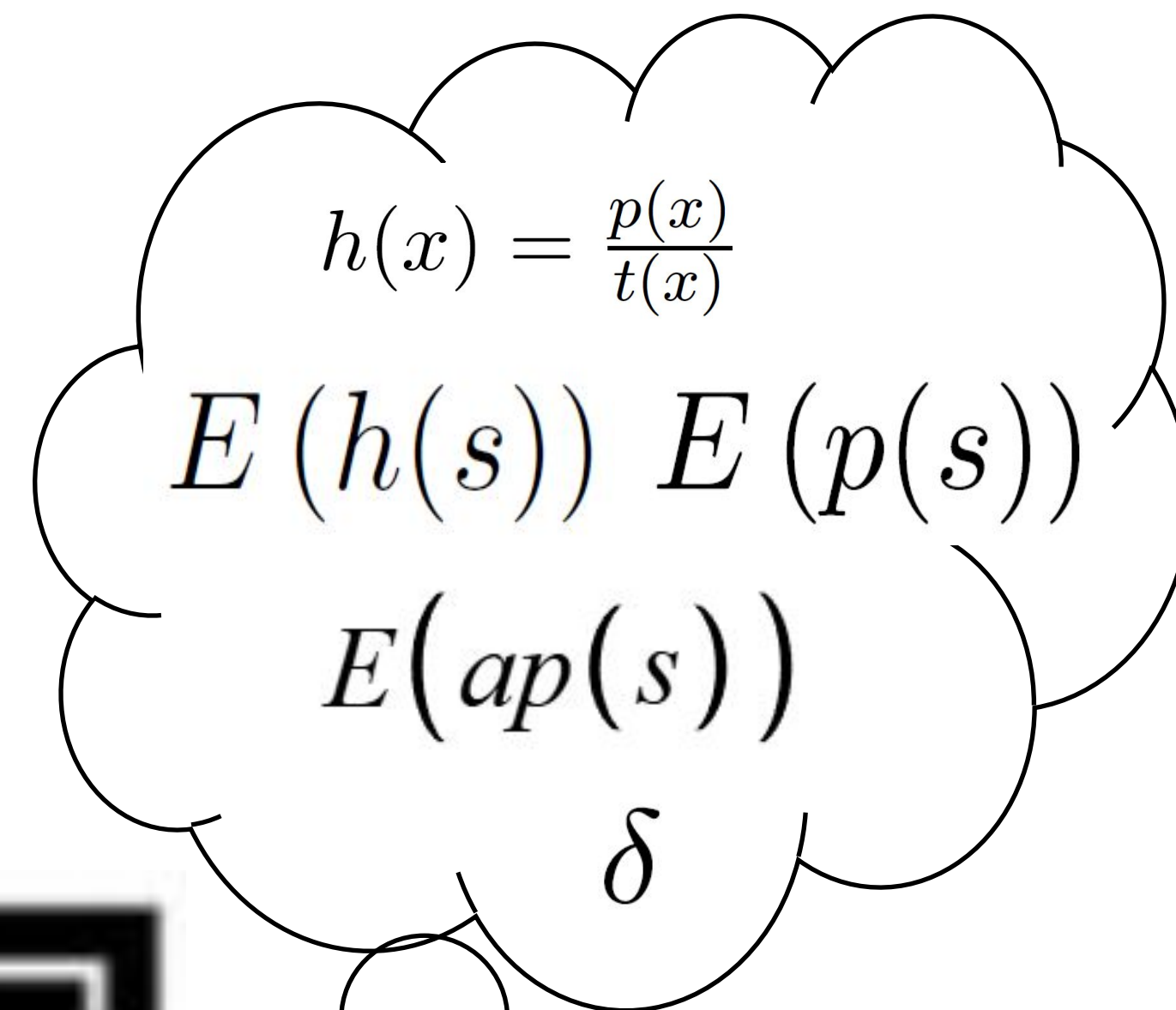


Bruce

$E[\alpha \cdot s^i] \quad E(s^i)$

$(g^{\delta p(s)}, g^{\delta h(s)}, g^{\delta \alpha p(s)})$

$(g^p, g^h, g^{p'})$



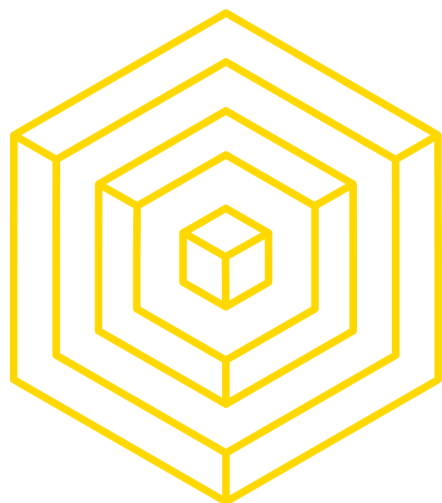
Jekyll

$$e(g^{p'}, g) = e(g^p, g^\alpha) \quad \checkmark$$

$$e(g^p, g) = e(g^{t(s)}, g^h) \quad \checkmark$$

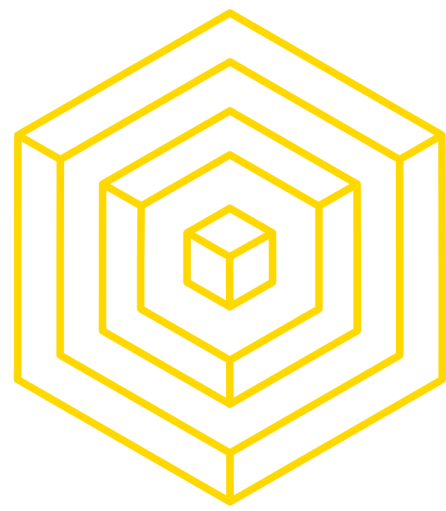


BLOCKCHAIN
AT BERKELEY



3

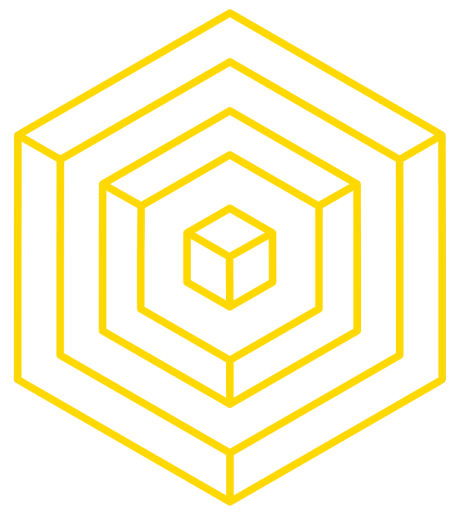
Rollups



Main Problem

- Transaction rates on the blockchain were way too slow
 - Bitcoin - 7 tx/sec
 - Ethereum - 15 tx/sec
- We need to speed these up to make blockchains faster while maintaining low tx fee
- How can we process txs faster?
 - Using faster consensus protocols
 - Parallelize processing by splitting the chain (sharding) i.e. Monad
 - Rollups (the modern solution)

Source: <http://cs251.stanford.edu>



Rollups

- The main idea of rollups is to batch many transactions together into one
- Hundreds of transactions on the rollup chain are batched into a single L1 transaction
- The rollup contract on L1 will hold assets/state of all rollup account

Source: <http://cs251.stanford.edu>



Rollups

ZK ROLLUPS



zkTube



STARKWARE



DeversiFi

dYdX

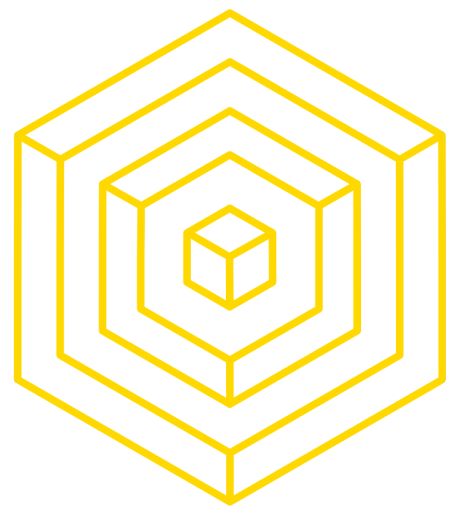


Aztec



zkSync



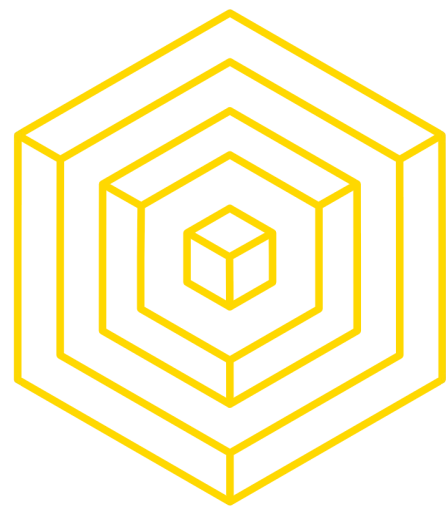


Rollups



Optimism





Problem #1 - Consensus on Rollups

- Rollup does not have a consensus protocol as it relies on the L1 chain to attest its state
- It uses a coordinator to maintain this state of all contracts on the protocol
- How can we make sure the coordinator is honest?
 - Validity proofs (zero knowledge rollups)
 - Fraud proofs (optimistic rollups)

Source: <http://cs251.stanford.edu>



Approach #1 - Validity Proofs

- Validity proofs use SNARK proofs, which are succinct proofs that validate hundreds of Tx's
- The rollup coordinator will send this SNARK proof along with the updated state root & tx list so that the L1 can prove it is valid and accept it
- SNARK proofs are fast and easy to verify on L1, proving that:
 - All user sigs on the tx are valid
 - All Merkle proofs are valid
 - Post state is the result of applying Tx list to pre-state
- Popular examples - StarkNet

Source: <http://cs251.stanford.edu>



- zkEVM - SNARK proof built by coordinator after successful execution of an EVM program, has two types:
 - Prove that EVM bytecode ran correctly (Polygon zkEVM, Scroll)
 - Compile Solidity to SNARK-friendly circuit (MatterLabs)
- The end result of the rollup contract on L1:
 - Ensures all Tx submitted have been signed by users
 - Ensures all state updates are valid
- It is also important to note that:
 - Rollup contract on L1 will accept any update with a valid proof
 - Anyone can act as a coordinator if they have enough compute power

Source: <http://cs251.stanford.edu>



Approach #2 - Fraud Proofs

- Fraud proofs, also called optimistic rollups, involve the coordinator depositing stake in escrow on the L1 rollup contract
 - Coordinator will submit state updates to L1 without a proof
 - If state update is invalid, there is a week period in which anyone can submit a fraud proof
 - If valid, the coordinator gets slashed on L1
 - If invalid, the complainer gets fined
 - Proving fraud is challenging because it requires re-executing all txs in batch
 - Another method is to binary search the Merkle tree
 - Major downside - 7 day finality delay

● Popular examples - Optimism

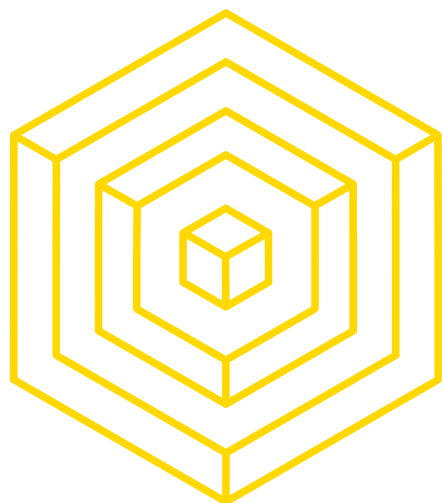
Source: <http://cs251.stanford.edu>



Problem #2 - Coordinator Dies

- If the coordinator dies, where do we get the latest rollup state to recover it from?
 - Rollup state can always be reconstructed from data on the L1 chain
 - Anyone can become a coordinator and read all Rollup update msgs and re-execute txs
 - Rollups use L1 for storage
 - For ZK - send tx summary without user sigs (since SNARK proves this)
 - For Optimistic - send tx summary and signatures
 - However, storing all of this is really expensive, so is there a better way?
 - Store tx data with a Data Availability Committee (DAC), which is a set of nodes trusted to keep the data available

Source: <http://cs251.stanford.edu>



4 Other use cases



ZK Other Applications

- Privacy-Preserving Transactions:
 - Verifying validity of transactions without revealing the transaction details. Very useful with financial applications
- Identity Verification:
 - Allow for identity verification without disclosing sensitive information. Individuals can prove they possess certain credentials without revealing those credentials
- Data Integrity Verification:
 - ZK proofs can be utilized to verify the integrity of data without exposing the data itself.
 - This can help with data validation and auditing
- Anonymous Systems:
 - ZK proofs enable the creation of anonymous systems where transactions or interactions can occur without revealing the identity of the parties involved

Source: <https://ethereum.org/en/developers/docs/scaling/zk-rollups/>



Rollup Other Applications

- Scalability in Blockchain:
 - Rollups allow a large number of off-chain transactions to be bundled into a single on-chain transaction, reducing the load on the main blockchain
- Reduced Gas Fees:
 - By aggregating transactions off-chain and submitting only a succinct proof on-chain, on-chain computation is reduced along with gas fees
- Smart Contract Execution:
 - Rollups can be used to execute smart contracts off-chain

Source: <https://ethereum.org/en/developers/docs/scaling/zk-rollups/>



Aleo

- Aleo is a blockchain platform that employs zk-SNARKs to achieve privacy in transactions and smart contracts
- How it works:
 - When users conduct transactions, ZK proofs allow them to prove the validity of the transaction without disclosing specific details
 - Aleo provides a developer-friendly environment for people to create efficient DApps
 - Aleo has its own native token, allowing for use cases such as transaction fees, staking, or governance
 - Aleo also focuses on interoperability, allowing assets and information to move seamlessly between different blockchain networks



Attendance/Vitamin

Code: ZK

